

1. 大模型原理与算力显存分析

核心知识点

Attention 演进：MHA $\rightarrow$ GQA $\rightarrow$ MQA $\rightarrow$ MLA (Multi-head Latent Attention)。重点理解 MLA 的低秩投影 (KV Compression) 如何大幅降低 KV Cache 显存占用。

MoE 核心机制：Top-k Router 路由算法、负载均衡 Loss、Token Drop 策略。

显存与计算模型（重点项）：

显存占用计算：模型权重 (FP16/BF16/FP8)、Optimizer States (Adam: $12\times$ 权重)、Gradients、Activation Memory、KV Cache 显存公式。

计算量估算：Flops 计算（例如前向传播约为 $2N$ FLOPs/token，反向约为 $4N$ FLOPs/token）。

推荐实践 / 深度剖析

推导一次标准 Transformer Layer 在 Sequence Length 为 L 、Hidden Size 为 H 时的显存分布与 FLOPs。

2. 大模型训练框架

核心知识点

DeepSpeed：

ZeRO-1/2/3 显存切分原理 (Optimizer State, Gradient, Parameter Partitioning)。

ZeRO-Offload (CPU/NVMe 卸载) 与 ZeRO-Infinity。

Megatron-LM：

底层张量并行与流水线并行的代码实现结构。

Sequence Parallelism (SP) 与 Selective Activation Checkpointing。

Ray：

Actor 与 Task 概念、Distributed Object Store (Plasma)。

Ray Train / Ray Data 在大模型集群调度的应用（特别是在 RLHF/PPO 训练流水线中的角色）。

推荐实践

基于 DeepSpeed ZeRO-3 手动写一个微调 7B 模型的脚本，并开启 Activation Checkpointing 观察 Peak Memory 变化。

3. 大模型推理框架

核心知识点

内存与调度优化：

KV Cache 管理：PagedAttention 原理（借鉴操作系统虚拟内存/页表思想，解决显存碎片）。

Dynamic Batching：Continuous Batching (Iteration-level batching) 与 Static Batching 的区别。

PD 分离 (Prefill & Decode Separation)：理解 Prefill (Compute-bound) 与 Decode (Memory-bound) 的瓶颈差异，为什么将它们拆分到不同节点上能提升吞吐与降低延迟。

框架源码剖析：

从 nano-vLLM 学习 PagedAttention 的 Python/CUDA 模拟实现。

从 Mini-SGLang 学习 RadixAttention (Prefix Caching 树状结构管理与复用) 机制。

推荐实践

用 vLLM 和 SGLang 跑通 Benchmark，对比 Prefill-heavy 和 Decode-heavy 负载下的 TPOP (Time per Output Token) 和 TTFT (Time to First Token)。

4. 分布式并行技术与架构设计

核心知识点

四大并行模式适用场景：

数据并行 (DP / DDP / FSDP)：适合模型单卡能放下，或结合其他并行度拓展卡数。

张量并行 (TP)：适合单卡放不下大模型，且跨卡通信带宽极高（如 NVLink）的场景（通常限制在单机 8 卡内）。

流水线并行 (PP): 适合跨机分布式训练, 掩盖通信延迟 (1F1B、Interleaved 1F1B 调度算法)。

专家并行 (EP): MoE 专属, 负责将不同路由的 Token 分发给不同的专家卡。

💡 开放题解题思路 (面试高频)

场景题: 假设给你 128 张 H800 (8卡/机, NVLink 带宽 900GB/s, 机间 InfiniBand 400Gbps), 如何训练一个 70B 或 236B MoE 模型?

解题切入点:

计算模型参数量与单卡显存配额。

单机内 (NVLink 范围内) 优先部署 **TP** 或 **EP** (高频通信需求)。

跨主机间部署 **PP** 和 **DP/ZeRO** (容忍较低通信带宽)。

评估 Bubble Time (PP 气泡率) 并调整 Micro-batch Size。

5. AI 算子开发与高性能计算

核心知识点

CUDA 基础: Thread, Block, Grid 层次结构; Shared Memory, Global Memory, Registers 存储层级; Warp Divergence (线程分支分化)。

优化思路深度:

访存优化: Coalesced Memory Access (合并访存)、Shared Memory Bank Conflict (利用 **Swizzle** / Padding 消除冲突)。

计算优化: Loop Unrolling、Instruction-level Parallelism (ILP)、Tensor Core (WMMA/MMA 指令) 调优。

异步数据搬运: Pipeline 机制 (利用 CUDA Streams 掩盖 H2D/D2H 及 Global → Shared 传输延迟)。

算子融合 (Kernel Fusion): 减少 Memory R/W 开销 (如 RMSNorm + Rearrange + GEMM 融合)。

性能分析工具:

Roofline 模型: 区分算子是 **Memory-bound** 还是 **Compute-bound**。

NCU (NVIDIA Nsight Compute): 分析 Memory Throughput, SOL (Speed of Light), Warp State 等指标。

经典算子实现原理:

Online Softmax: 消除传统 Softmax 两次 Read/Write 开销的数学归纳推导。

FlashAttention (v1/v2/v3): Tiling 分块计算、SRAM 缓存调度、Backward 重算机制。

Triton: Pythonic 的 GPU 编程语言，了解 Block-level 编程范式。

6. 大模型量化技术

核心知识点

量化加速本质:

Memory-bound 阶段 (Decode): 低位量化 (INT4/INT8/FP4) 大幅减少读写内存带宽 (BW)，提升每秒读取的 Token 数量。

Compute-bound 阶段 (Prefill): 需依托硬件 INT8/FP8 Tensor Core 计算单元进行实际算力加速。

主流算法:

GPTQ: 基于二阶导数 (Hessian 矩阵) 的 Post-Training Quantization (PTQ)。

AWQ: 保护 1% 激活值 (Salient Weights) 保留表达能力，对剩余权重按 Channel 缩放量化。

SmoothQuant: 将激活值的 Outliers 难度通过 mathematically-equivalent 方式转移给 Weight。

Marlin: 针对 4-bit 量化 GEMM 的极致优化 CUDA Kernel (针对 Memory-bound 场景优化了 Dequantization 步骤)。

7. 强化学习 (RL / RLHF)

核心知识点

传统 RL 基础 (参考《动手学强化学习》):

MDP (Markov Decision Process)、Value-based (DQN)、Policy-based (REINFORCE)、Actor-Critic (PPO)。

LLM 领域的 RLHF / Alignment:

PPO 原理: Reward Model、Actor Network、Critic Network、Reference Model 4 个模型协调训练的显存调度与通信开销。

DPO (Direct Preference Optimization): 无需显式 Reward Model 的隐式偏好优化数学证明。

GRPO / PPO-Free 技术：大模型（如 DeepSeek-R1）推理强化学习调优中的近况。

8. 集合通信（Collective Communication）

核心知识点

原语实现与通信量计算：

Broadcast / Reduce

All-Reduce（Ring-AllReduce 算法推导：通信量为 $2 \times \frac{N-1}{N} \times S$ ）

All-Gather（通信量为 $\frac{N-1}{N} \times S$ ）

Reduce-Scatter（通信量为 $\frac{N-1}{N} \times S$ ）

All-to-All（MoE 专家调度的核心通信算子）

拓扑结构与硬件拓扑：

Ring 算法 vs. Tree 算法在不同规模下的效率差异。

NCCL (NVIDIA Collective Communications Library) 内部 Ring 构建与 NVLink/InfiniBand 拓扑感知。

💡 总结建议与进阶学习路线阶段规划

阶段	核心任务	交付物 / 检验标志
第一阶段：基建与手撕基础	深入理解 CUDA 内存架构，学习 FlashAttention 原理	手写 CUDA/Triton 实现 Naive Attention 及 Vector Add；跑通 NCU 性能瓶颈分析。
第二阶段：推理端进阶	读懂 nano-vLLM / Mini-SGLang 源码	实现一个带 PagedAttention 和 Dynamic Batching 的简易推理 Server。

阶段

核心任务

交付物 / 检验标志

第三阶段：训练与分布式系统

熟悉 Megatron-LM / DeepSpeed

能够推导 70B 模型在不同 DP/TP/PP/ZeRO 组合下的显存预算与通信耗时。

